

Projekt: StudyConnect

Lab 7

SAST Scan

Setup

Run once to install Semgrep

```
pip install semgrep
```

Scan repository

```
semgrep --config=p/security-audit \--config=p/owasp-top-ten \--config=p/cwe-top-25 \--json - \
--output=sast-results.json \.
```

Categorization of findings

CWE Classification

Based on the Semgrep scan results, the identified findings can be categorized into the following CWE categories:

Category	CWE	Description
Authentication Failures / Privilege Management	CWE-250	Missing non-privileged user definition in Dockerfiles
Exposure	CWE-668	Binding the Flask application to all network interfaces (0.0.0.0)
Exposure	CWE-489	Hardcoded debug/test configuration (TESTING = True) in test files

SAST Findings Table & Triage

File	Line	Rule / Vulnerability	CWE Category	Severity	Triage Decision	Justification
backend/Dockerfile	19	Missing User	Authentication Failures / Privilege Management (CWE-250)	ERROR	True Positive	The container runs as the root user by default. This represents a genuine security risk, and a non-privileged user should be defined.
ui/Dockerfile	33	Missing User	Authentication Failures /	ERROR	True Positive	This container also runs as

File	Line	Rule / Vulnerability	CWE Category	Severity	Triage Decision	Justification
			Privilege Management (CWE-250)			root. Nginx can and should be configured to run under a non-privileged user (e.g., <code>nginx</code>).
<code>backend/api.py</code>	456	Bind to <code>0.0.0.0</code>	Exposure (CWE-668)	WARNING	Acceptable Risk	In a Docker environment, binding to <code>0.0.0.0</code> is necessary to make the container accessible. Since Gunicorn is correctly used in production, this is considered an acceptable risk within the Flask script.
<code>backend/features/environment.py</code>	27	Hardcoded <code>TESTING=True</code>	Exposure (CWE-489)	WARNING	False Positive	This is a dedicated test file (Behave environment configuration). Enabling testing mode here is necessary and legitimate.
<code>test_backend/test_api.py</code>	56	Hardcoded <code>TESTING=True</code>	Exposure (CWE-489)	WARNING	False Positive	This is also test code (Pytest fixtures). While the warning is valid for production code, it does not apply to test files.

Cross Check: SAST vs. Threat Modeling (Lab 5) Assessment

By comparing the top 8 critical risks identified during our Threat Modeling in Lab 5 with the findings from the Semgrep SAST scan, we can evaluate the effectiveness and limitations of Static Application Security Testing:

Threats Detected by SAST

- **R-08: Debug mode or verbose error exposure:**

- *SAST Finding:* CWE-489 (Hardcoded `TESTING=True` in test files).
- *Evaluation:* Semgrep successfully looked for this exact vulnerability category. Even though it was a false positive in this specific context (since the code resides in test files), it proves the scanner actively protects against debug configurations leaking into production.

Threats not Detected by SAST (and why)

Static analysis tools inherently struggle with complex business logic. The following risks were not detected:

- **R-02 (IDOR), R-03 (Mass Assignment), R-04 (Group-member disclosure), R-05 (Covert group join):**

- *Reason:* These are Business Logic Flaws. SAST tools only analyze code syntax and data flow. They cannot "understand" that a `user_id` from a request must match the `user_id` of the database record. Detecting these requires specific custom rules, manual review, or functional security testing (DAST).

- **R-07: Hardcoded or exposed secrets:**

- *Reason:* Our secrets are primarily stored in `.env` files or `docker-compose.yml`. Semgrep's Python ruleset focuses on source code. This confirms the need for a dedicated Secret Scanning tool (like GitLeaks or TruffleHog), as proposed in Lab 2.

- **R-06: Registration flooding / brute-force requests:**

- *Reason:* Rate limiting is typically implemented at the infrastructure level (e.g., API Gateway, Nginx) or via specific framework configurations, which standard SAST rules rarely verify effectively.

- **R-01: Token theft from browser storage / XSS session hijack:**

- *Reason:* While SAST is generally good at finding XSS, Semgrep skipped several advanced JavaScript rules for the React frontend because the Open-Source (OSS) engine lacks certain taint-tracking features required by those specific OWASP rules. Additionally, storing tokens in `localStorage` is often considered a contextual architectural flaw rather than a strict syntax error.

Unanticipated Findings (beyond Lab 5)

- **CWE-250 (Execution with Unnecessary Privileges in Docker):**

- The SAST scan identified a critical security risk (containers running as `root`) that was not part of the Lab 5 Threat Model. The Threat Model focused heavily on application data flows (APIs, JWTs, DB) but overlooked infrastructure and container privileges.

Manual Security Code Review

As demonstrated above, SAST tools fail to catch business logic and authorization flaws. A manual security code review identified the following high-risk areas that require remediation:

Finding	Category	Severity	SAST caught?	Fix
IDOR at GET <code>/api/tasks/user/<user_id></code> Any authenticated user can read the tasks of any other user.	Authentication & Authorization	Critical	No	Implement resource-level check: verify <code>request.user['sub'] == user_id</code> .
IDOR at GET <code>/api/groups/user/<user_id></code> Foreign group memberships can be read without restriction.	Authentication & Authorization	High	No	Enforce match of token ID with requested URL parameter ID.
Logic flaw at GET <code>/api/groups/user/admin/<user_id></code> Token is validated only if the user does <i>not</i> exist in the database yet.	Business Logic	High	No	Perform strict authorization BEFORE any database access.
Exposure of PII at GET <code>/api/users/<user_id></code>	Data Protection	Medium	No	Restrict access to own profile or hide sensitive data for other users.

Finding	Category	Severity	SAST caught?	Fix
Foreign user profiles (incl. email, birthday, faculty) are publicly accessible.				

The Role of Cognitive Biases in Manual Reviews

While manual reviews are crucial for catching business logic flaws, human reviewers are still susceptible to overlooking vulnerabilities due to cognitive biases. Common pitfalls include:

- **Author Bias / Confirmation Bias:** Reviewers examining their own code tend to see what they intended the code to do, mentally skipping over logic flaws because they assume the underlying concept is sound ("I wrote this, so I know how it works").
- **Automation Bias:** Over-relying on automated tools (like SAST) or AI assistants can lead to a false sense of security. If the tool reports no errors, reviewers might subconsciously lower their guard ("The scanner didn't complain, so it must be secure").
- **Halo Effect / Authority Bias:** Blindly trusting peers or senior developers reduces the rigor of the review. Reviewers might assume the author already considered security implications ("My teammate is a great developer, I'm sure this is correct").

Comparison: SAST vs. Manual Review

Vulnerability Category	SAST Scan (Semgrep)	Manual Security Code Review
Infrastructure & Configuration	Caught: Docker missing non-root user (CWE-250) Caught: Flask bound to 0.0.0.0 (CWE-668)	Missed (Review focused on application logic rather than Dockerfiles)
Debug & Hardcoded Secrets	Caught: Hardcoded TESTING=True in test files (CWE-489)	Missed (Often overlooked by humans due to context switching)
Authorization & IDOR	Missed (Scanners cannot understand custom resource ownership or business rules)	Caught: Critical IDORs on /api/tasks/user/<user_id> and /api/groups/user/<user_id>
Business Logic Flaws	Missed (Syntax was correct, so the scanner saw no issue)	Caught: Flawed token validation logic on /api/groups/user/admin/<user_id>
Data Protection / PII	Missed (Scanners do not inherently know what data is considered sensitive PII)	Caught: Public exposure of sensitive profile data on /api/users/<user_id>
Primary Strengths	Highly scalable, catches misconfigurations, syntax errors, and infrastructure flaws instantly.	Deep contextual awareness; understands intended permission models, roles, and business rules.
Primary Weaknesses	Prone to false positives (e.g., flagging test files); completely blind to business logic.	Slow, not scalable, and susceptible to human cognitive biases (e.g., automation bias, review fatigue).

Gaps (Tool Blind Spots vs. Human Context)

When securing software, a fundamental gap exists between automated analysis and manual review.

What would only tools catch?

- **Scale and Fatigue-based Errors:** Tools can scan millions of lines of code, complex dependency trees, and thousands of configuration files in seconds without losing focus.
- **Known Patterns & Syntax:** Scanners excel at finding hardcoded secrets via regex, identifying known CVEs in third-party libraries, and spotting syntax-level misconfigurations (e.g., missing security headers, weak cryptographic algorithms).

What would only humans catch?

- **Business Logic & Authorization Flaws:** Only a human understands the intended permission model. A tool cannot know that user A shouldn't be able to view user B's tasks (IDOR), or that an admin token validation is implemented backward.
- **Contextual Data Sensitivity:** A scanner sees a JSON string being returned. A human reviewer understands the domain context and knows that returning a **birthday** field on a public profile violates privacy requirements (PII exposure).

Top 3 Findings & Proposed Fixes

1. Insecure Direct Object Reference (IDOR) on Task Endpoints
 - see table "Manual Security Code Review"
2. Logic flaw at token validation on **/api/groups/user/admin/<user_id>**
 - see table "Manual Security Code Review"
3. Missing Non-Privileged User Definition in Dockerfiles
 - Fix: Modify the Dockerfiles to create a dedicated, unprivileged system user and group during the build process. Use the **USER** instruction in the Dockerfile to switch execution from **root** to this unprivileged user before the container's entrypoint command is executed.

Executive summary

Although the StudyConnect codebase appears fundamentally solid, the current security posture requires immediate attention due to critical vulnerabilities in infrastructure configuration and application business logic. While automated SAST scanning successfully identified privilege management risks like Docker containers running as root, manual code reviews uncovered severe authorization flaws such as IDORs and public exposure of sensitive information. To achieve a secure and CRA-compliant state, we must remediate these high-risk findings.